

Question Bank — Week 9

Basic sorting (Lecture 09) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	2
Part B — Short answer and essay	5
Part C — Trace the algorithms	5
Part D — Array state	6
Part E — Complexity analysis	6
Part F — Find and fix the bug	6
Part G — Write the code	8

Answers are in a separate file: [week09-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. The sorts are those of `dsa/sorting.py`: **bubble sort** compares neighbours and stops after a pass with no swap; **selection sort** swaps the minimum of `a[i..n-1]` into slot `i`, skipping the swap when it is already there; **insertion sort** shifts every larger value right and drops current into the hole, walking left while `j >= 0` and `a[j] > current`; `counting_sort` counts the values `0..max`. A **pass** of bubble sort and a **round** of the other two are one turn of their outer loop. Unless a question says otherwise, **A** is the lecture's list

A = [5, 2, 9, 1, 7, 3]

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W9-M01 – W9-M22
B	Short answer and essay	W9-E1 – W9-E5
C	Trace the algorithms	W9-T1 – W9-T5
D	Array state — draw every step	W9-S1 – W9-S3
E	Complexity analysis	W9-K1 – W9-K3
F	Find and fix the bug	W9-B1 – W9-B4
G	Write the code — checked by pytest	W9-C1 – W9-C5

Part A — Multiple choice

W9-M01 [what] A sorting algorithm is **stable** when:

- **a)** it uses no memory besides the array
- **b)** it takes the same time on every input of size n
- **c)** elements with equal keys keep their original relative order
- **d)** it makes at most $n - 1$ swaps

W9-M02 [what] A sort is **in place** when:

- **a)** it needs only $O(1)$ extra memory besides the array it sorts
- **b)** it returns the same list object it was given
- **c)** it is stable
- **d)** it runs in $O(n)$ time

W9-M03 [what] Which of these sorts is **not** stable, as written in the course?

- **a)** bubble sort
- **b)** insertion sort
- **c)** merge sort that takes from the left half on a tie
- **d)** selection sort

W9-M04 [how] Bubble sort, with the early exit, on $[1, 2, 3, 4, 5, 6]$ makes how many comparisons?

- **a)** 5
- **b)** 15
- **c)** 6
- **d)** 36

W9-M05 [how] Selection sort on $[1, 2, 3, 4, 5, 6]$ makes how many comparisons?

- **a)** 5
- **b)** 6
- **c)** 15
- **d)** 0

W9-M06 [how] The array after the **first pass** of bubble sort on A is:

- **a)** $[1, 2, 9, 5, 7, 3]$
- **b)** $[2, 5, 9, 1, 7, 3]$
- **c)** $[2, 1, 5, 3, 7, 9]$
- **d)** $[2, 5, 1, 7, 3, 9]$

W9-M07 [how] The array after the **first round** of selection sort on A is:

- **a)** $[2, 5, 1, 7, 3, 9]$
- **b)** $[1, 2, 9, 5, 7, 3]$
- **c)** $[1, 5, 2, 9, 7, 3]$
- **d)** $[1, 2, 3, 5, 7, 9]$

W9-M08 [how] Insertion sort on A. The array after the round that inserts a [3] (the value 1) is:

- **a)** [2, 5, 9, 1, 7, 3]
- **b)** [1, 2, 9, 5, 7, 3]
- **c)** [2, 1, 5, 9, 7, 3]
- **d)** [1, 2, 5, 9, 7, 3]

W9-M09 [how] How many **inversions** does [3, 1, 2] have?

- **a)** 1
- **b)** 3
- **c)** 2
- **d)** 0

W9-M10 [why] The number of shifts insertion sort makes is always equal to:

- **a)** $n - 1$
- **b)** the number of inversions of the input
- **c)** $n(n - 1)/2$
- **d)** $\lceil \log_2 n! \rceil$

W9-M11 [what] The best case of insertion sort is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n \log n)$
- **d)** $O(n)$, on sorted input

W9-M12 [why] Selection sort is the best of the three basic sorts when:

- **a)** the data is nearly sorted
- **b)** writing an element costs much more than comparing two
- **c)** the sort must be stable
- **d)** the values are small integers

W9-M13 [how] The largest number of swaps selection sort makes on 10 values is:

- **a)** 9
- **b)** 45
- **c)** 10
- **d)** 100

W9-M14 [how] `counting_sort([4, 0, 4, 1])` makes a counts array of how many slots?

- **a)** 4
- **b)** 5
- **c)** 3
- **d)** 2

W9-M15 [what] Counting sort on n values from the range $0 \dots k - 1$ takes:

- **a)** $O(n \log n)$

- **b)** $O(n^2)$
- **c)** $O(n + k)$
- **d)** $O(k \log n)$

W9-M16 [why] Counting sort is a **bad** choice when:

- **a)** the values are small integers
- **b)** n is large
- **c)** the range of the values is much larger than n
- **d)** the input is already sorted

W9-M17 [what] In the worst case, every comparison sort of n values needs at least:

- **a)** $\log_2 n$ comparisons
- **b)** $\log_2 n!$ comparisons
- **c)** n^2 comparisons
- **d)** $n/2$ comparisons

W9-M18 [how] The fewest comparisons that can sort **any** 4 values in the worst case, by comparing, is at least:

- **a)** 4
- **b)** 6
- **c)** 3
- **d)** 5

W9-M19 [why] The decision-tree argument for the lower bound uses the fact that:

- **a)** the tree needs at least $n!$ leaves, and a binary tree of height h has at most 2^h leaves
- **b)** every sort is recursive
- **c)** every comparison halves the array
- **d)** the tree is always balanced

W9-M20 [why] A `_steps` generator yields its working **list itself** (not a copy) before each comparison. Then `frames = list(generator)` holds:

- **a)** the final, sorted state in every frame
- **b)** a `TypeError`
- **c)** the right frames
- **d)** only the first frame

W9-M21 [why] A plain sort runs its `_steps` generator to the end, and the generator copies the array before each of its about $n^2/2$ comparisons. The plain sort costs:

- **a)** $O(n^2)$
- **b)** $O(n^3)$
- **c)** $O(n \log n)$
- **d)** $O(n)$

W9-M22 [how] Bubble sort on `[6, 5, 4, 3, 2, 1]` makes how many swaps?

- **a)** 5
- **b)** 6

- c) 15
 - d) 36
-

Part B — Short answer and essay

W9-E1 [why] (4 marks) Define **stable** and **in place**. Say which of bubble, selection and insertion sort are stable, and why, from their code. Give a small input on which selection sort is not stable, and explain why stability matters when you sort by two keys.

W9-E2 [how] (4 marks) Compare bubble, selection and insertion sort: the comparisons they make at best and at worst, and the swaps or shifts. For each, describe a situation in which it is the best of the three.

W9-E3 [why] (4 marks) Define an **inversion**. Prove that insertion sort makes exactly one shift per inversion, and use that to give its best, worst and average case. Why do library sorts such as Timsort use insertion sort inside?

W9-E4 [why] (4 marks) Explain the `_steps` generator pattern of `dsa/sorting.py`: what it yields and when, how the plain sort reuses it, the two rules its frames must obey, and what “the price of watching” is and how it is fixed.

W9-E5 [why] (4 marks) State the lower bound for comparison sorting and give the decision-tree argument for it. Explain why counting sort does not contradict it, and when counting sort is the right choice.

Part C — Trace the algorithms

W9-T1 [how] Trace bubble sort (with the early exit) on `[3, 8, 1, 6, 2]`: the array after each pass, and the comparisons and swaps of each pass.

W9-T2 [how] Trace selection sort on `[29, 10, 14, 37, 13]`: for each round, the index of the minimum, whether it swaps, and the array after the round.

W9-T3 [how] Trace insertion sort on `[8, 4, 6, 2, 9, 1]`: the array after each round, with its shifts and comparisons. Check the total shifts against the number of inversions.

W9-T4 [why] How many frames does each reference `_steps` generator yield on `[5, 4, 3, 2, 1]` — bubble, selection and insertion? Explain each count from where the `yield` statements are.

W9-T5 [how] Trace `counting_sort([2, 5, 3, 0, 2, 3, 0, 3])`: `k`, the counts array, and the output. Then give the **starting positions** that the stable version (W9-C5) computes from the counts, and say where the first 3 and the last 3 of the input are placed.

Part D — Array state

W9-S1 [how] Draw the array after every pass of bubble sort on [6, 5, 3, 1, 8, 7, 2, 4], shading the values that are in their final place. How many passes run, and how many comparisons and swaps in total?

W9-S2 [how] Draw every frame that the reference `insertion_sort_steps` yields on [5, 1, 4, 2, 8], with its highlight. Why does each frame show a permutation of the input — no value twice?

W9-S3 [how] The Dutch national flag partition (W9-C3) keeps three indices: `values[:lo]` are 0, `values[lo:mid]` are 1, `values[hi+1:]` are 2, and `values[mid..hi]` is not yet seen. At each step it looks at `values[mid]`: a 0 is swapped to `lo` (both advance), a 1 is left (`mid` advances), a 2 is swapped to `hi` (`hi` retreats, `mid` stays). Starting from `lo = mid = 0`, `hi = 5`, draw the array and the three indices after each step on [2, 0, 2, 1, 1, 0].

Part E — Complexity analysis

W9-K1 [how] Give the best and worst case, in Θ , and justify:

```
def has_duplicates(values):
    s = insertion_sort(values)
    for i in range(len(s) - 1):
        if s[i] == s[i + 1]:
            return True
    return False
```

What would it be with `counting_sort`, for values in $0 \dots k - 1$? And with two nested loops comparing every pair?

W9-K2 [why] Derive the exact number of comparisons selection sort makes on n values, and the largest number of swaps. Measured on $n = 1,000$ random values, selection sort made 499,500 comparisons and 1,986 writes, and insertion sort 249,614 comparisons and 498,241 writes. If a write costs 100 times as much as a comparison, which is cheaper, and by how much?

W9-K3 [why] Every value of a list is at most k places from its sorted position. Show that insertion sort sorts it in $O(nk)$. What does that give for $k = 1$, and for $k = n$?

Part F — Find and fix the bug

W9-B1 [how]

```
def bubble_sort_steps(values):
    a = Array.from_values(values)
```

```

n = len(a)
yield list(a), ()
for end in range(n - 1, 0, -1):
    swapped = False
    for j in range(end + 1):
        yield list(a), (j, j + 1)
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            swapped = True
    if not swapped:
        break
yield list(a), ()

```

W9-B2 [why] It sorts, and it passes all twelve selection tests. What is wrong with it? Count its swaps on [6, 5, 4, 3, 2, 1].

```

def selection_sort(values):
    a = Array.from_values(values)
    n = len(a)
    for i in range(n - 1):
        for j in range(i + 1, n):
            if a[j] < a[i]:
                a[i], a[j] = a[j], a[i]
    return list(a)

```

W9-B3 [why] It passes every test except one. Which one, and why?

```

def insertion_sort(values):
    a = Array.from_values(values)
    for i in range(1, len(a)):
        current = a[i]
        j = i - 1
        while j >= 0 and a[j] >= current:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = current
    return list(a)

```

W9-B4 [how]

```

def counting_sort(values):
    k = max(values)
    counts = Array(k, fill=0)
    for v in values:
        counts[v] += 1
    out, i = Array(len(values)), 0
    for v in range(k):

```

```
    for _ in range(counts[v]):  
        out[i] = v  
        i += 1  
    return list(out)
```

Part G — Write the code

In `practice/week09.py`; check with `pytest tests/test_practice_week09.py -v`. Do not call `sorted`, `list.sort`, `min` or `max` on the values. The tests count comparisons (C4) and reads and writes (C3), so the stated cost is checked.

W9-C1 [how] `count_inversions(values)` — the number of pairs in the wrong order, in $O(n^2)$. Why does it equal the number of swaps bubble sort makes?

W9-C2 [why] `sort_by_key(items, key)` — a stable insertion sort that compares only `key(item)`. Why must it not compare the items themselves?

W9-C3 [why] `dutch_flag(values)` — sort 0s, 1s and 2s in place in one pass. Why does `mid` not advance after swapping a 2 to the back?

W9-C4 [how] `sort_k_sorted(values, k)` — every value is at most k places from home; sort in $O(nk)$.

W9-C5 [why] `counting_sort_by_key(items, key, k)` — a **stable** counting sort of records, $O(n + k)$, with no comparisons. Why must the placing loop go through the items in input order?